

# LABORATORY OBSERVATION / RECORD

Course: Full Stack Web Development

Student Name: Y.Abhishek

Course Code: 23CM4121

Roll No.: A24126552127

Experiment No.: 4

Date: 18 – 08 – 2026

## 1. TITLE

Develop a To-Do List Application Using JavaScript DOM Manipulation and Event Listeners

## 2. AIM

To design and develop a dynamic To-Do List web application that uses JavaScript DOM manipulation to dynamically create, complete, and delete tasks, and to conditionally display an empty-state message.

## 3. OBJECTIVE

The objectives of this experiment are:

- To understand and use `document.createElement()` to dynamically build DOM elements.
- To append and remove child nodes using `appendChild()` and `remove()`.
- To handle user interaction using `addEventListener()` for click and keydown events.
- To capture and validate text input, trimming whitespace and preventing empty entries.
- To toggle CSS classes dynamically using `classList.toggle()` for visual state changes.
- To conditionally show or hide an element using `style.display` based on list state.
- To style task items, buttons, and completed states using CSS3 Flexbox and transitions.

## 4. THEORY

**DOM Element Creation** — `document.createElement()` constructs a new, detached DOM element in memory. Properties such as `textContent` assign visible text, and `className` assigns CSS styling classes for structured presentation.

**DOM Tree Manipulation** — `appendChild()` attaches a child node as the last descendant of a parent element, enabling dynamic runtime construction of list items. The `remove()` method permanently detaches an element from the active document tree.

**Event Listeners** — `addEventListener()` binds callback functions to user interaction events (click, keydown), providing interactive, event-driven functionality without inline HTML handlers.

`classList.toggle()` — Toggles a specified CSS class on an element, adding it if absent and removing it if present. This enables clean binary state transitions such as toggling a task between active and completed.

**CSS Flexbox Layout** — `display: flex` aligns task description text and action buttons along a single horizontal axis with space-between distribution, ensuring clean visual alignment across varying text lengths.

## 5. ALGORITHM / PROCEDURE

1. Create three project files: `todo.html`, `style.css`, and `script.js`.

2. In `todo.html`, construct the container (`<main class="todo-container">`) with heading 'My To-Do List', a text input field, an 'Add Task' button, an empty `<ul id="taskList">`, and an empty-state `<p id="emptyMessage">`.
3. Link `style.css` in the `<head>` and `script.js` before the closing `</body>` tag.
4. In `script.js`, select the input field, add button, task list, and empty message elements using `getElementById()`.
5. Write `updateEmptyState()` to show `emptyMessage` ('No tasks available.') when `taskList` has no children, and hide it otherwise.
6. Write `addTask()` to read and trim the input value, validating that empty inputs are rejected.
7. Inside `addTask()`, dynamically create a `<li>` (task-item), a `<span>` (task-text), a `<div>` (task-actions), a Complete `<button>`, and a Delete `<button>` using `createElement()`.
8. Attach a click listener to the Complete button to toggle class `completed` on the `<li>` and toggle button label between 'Complete' and 'Completed'.
9. Attach a click listener to the Delete button to remove the `<li>` from the DOM using `remove()` and call `updateEmptyState()`.
10. Assemble the created nodes using `appendChild()`, append the `<li>` to `taskList`, clear the input field, and call `updateEmptyState()`.
11. Attach click listener on Add Task button and keydown listener on `taskInput` to trigger `addTask()` on Enter key press.
12. Call `updateEmptyState()` on page load to display the initial 'No tasks available.' message.
13. Open `todo.html` in a browser, add multiple tasks, mark tasks complete, delete tasks, and verify empty-state behavior.

## 6. PROGRAM

### `todo.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My To-Do List</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <main class="todo-container">
    <!-- Application Header -->
    <header class="todo-header">
      <h1>My To-Do List</h1>
    </header>

    <!-- Task Input Controls -->
    <section class="input-section">
      <input type="text" id="taskInput" placeholder="Enter a task..." autocomplete="off">
      <button id="addTaskBtn" type="button">Add Task</button>
    </section>

    <!-- Task List Area -->
    <section class="task-section">
      <!-- Empty state message displayed when no tasks exist -->
      <p id="emptyMessage" class="empty-message">No tasks available.</p>

      <!-- Dynamic task items will be appended here -->
      <ul id="taskList" class="task-list"></ul>
    </section>
  </main>
```

```
<script src="script.js"></script>
</body>
</html>
```

## style.css

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: 'Segoe UI', system-ui, -apple-system, BlinkMacSystemFont, Roboto, sans-serif;
}

body {
  background: linear-gradient(135deg, #f8fafc 0%, #e2e8f0 100%);
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: flex-start;
  padding: 50px 20px;
  color: #1e293b;
}

/* Card Container */
.todo-container {
  background: #ffffff;
  width: 100%;
  max-width: 560px;
  border-radius: 14px;
  box-shadow: 0 10px 25px -5px rgba(0, 0, 0, 0.08), 0 8px 10px -6px rgba(0, 0, 0, 0.04);
  border: 1px solid #e2e8f0;
  padding: 30px;
}

/* Header */
.todo-header {
  text-align: center;
  margin-bottom: 24px;
  padding-bottom: 16px;
  border-bottom: 2px solid #f1f5f9;
}

.todo-header h1 {
  font-size: 1.75rem;
  font-weight: 700;
  color: #0f172a;
  letter-spacing: -0.02em;
}

/* Input Section */
.input-section {
  display: flex;
  gap: 12px;
  margin-bottom: 24px;
}

.input-section input {
  flex: 1;
  padding: 12px 16px;
  font-size: 0.95rem;
  color: #1e293b;
  border: 1.5px solid #cbd5e1;
```

```

    border-radius: 8px;
    outline: none;
    transition: all 0.2s ease;
}

.input-section input:focus {
    border-color: #3b82f6;
    box-shadow: 0 0 0 3px rgba(59, 130, 246, 0.15);
}

.input-section button {
    padding: 12px 22px;
    background-color: #2563eb;
    color: #ffffff;
    font-size: 0.95rem;
    font-weight: 600;
    border: none;
    border-radius: 8px;
    cursor: pointer;
    transition: background-color 0.2s ease, transform 0.1s ease;
    white-space: nowrap;
}

.input-section button:hover {
    background-color: #1d4ed8;
}

.input-section button:active {
    transform: scale(0.98);
}

/* Empty State */
.empty-message {
    text-align: center;
    color: #64748b;
    font-size: 0.95rem;
    font-style: italic;
    padding: 24px 16px;
    background-color: #f8fafc;
    border: 1px dashed #cbd5e1;
    border-radius: 8px;
    margin: 10px 0;
}

/* Task List */
.task-list {
    list-style: none;
    display: flex;
    flex-direction: column;
    gap: 10px;
}

/* Dynamic Task Item */
.task-item {
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: 12px 16px;
    background-color: #f8fafc;
    border: 1px solid #e2e8f0;
    border-radius: 8px;
    transition: all 0.2s ease;
}

```

```
.task-item:hover {
  background-color: #f1f5f9;
  border-color: #cbd5e1;
}

.task-text {
  flex: 1;
  font-size: 0.95rem;
  color: #1e293b;
  word-break: break-word;
  margin-right: 14px;
  transition: all 0.2s ease;
}

/* Completed State */
.task-item.completed {
  background-color: #f0fdf4;
  border-color: #bbf7d0;
}

.task-item.completed .task-text {
  text-decoration: line-through;
  color: #94a3b8;
}

/* Task Buttons Group */
.task-actions {
  display: flex;
  gap: 8px;
  flex-shrink: 0;
}

.btn-complete {
  padding: 6px 14px;
  background-color: #10b981;
  color: #ffffff;
  font-size: 0.82rem;
  font-weight: 600;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  transition: background-color 0.15s ease;
}

.btn-complete:hover {
  background-color: #059669;
}

.task-item.completed .btn-complete {
  background-color: #64748b;
}

.task-item.completed .btn-complete:hover {
  background-color: #475569;
}

.btn-delete {
  padding: 6px 14px;
  background-color: #ef4444;
  color: #ffffff;
  font-size: 0.82rem;
  font-weight: 600;
```

```

border: none;
border-radius: 6px;
cursor: pointer;
transition: background-color 0.15s ease;
}

.btn-delete:hover {
  background-color: #dc2626;
}

/* Responsive */
@media (max-width: 480px) {
  .todo-container {
    padding: 20px 16px;
  }

  .input-section {
    flex-direction: column;
  }

  .input-section button {
    width: 100%;
  }

  .task-item {
    flex-direction: column;
    align-items: flex-start;
    gap: 10px;
  }

  .task-actions {
    width: 100%;
    justify-content: flex-end;
  }
}

```

## script.js

```

/**
 * Task-4: To-Do List Application
 * Demonstrating DOM Selection, Dynamic Element Creation, and Event Handling
 */

document.addEventListener("DOMContentLoaded", () => {
  // 1. DOM Element Selection
  const taskInput = document.getElementById("taskInput");
  const addTaskBtn = document.getElementById("addTaskBtn");
  const taskList = document.getElementById("taskList");
  const emptyMessage = document.getElementById("emptyMessage");

  // 2. Helper function to toggle empty state message
  function updateEmptyState() {
    if (taskList.children.length === 0) {
      emptyMessage.style.display = "block";
    } else {
      emptyMessage.style.display = "none";
    }
  }

  // 3. Dynamic Element Creation & Task Addition
  function addTask() {
    const taskTextValue = taskInput.value.trim();
  }
}

```

```

// Validation: do not add empty tasks
if (taskTextValue === "") {
    alert("Please enter a task before adding!");
    taskInput.focus();
    return;
}

// Create <li> element for the task item
const li = document.createElement("li");
li.className = "task-item";

// Create <span> element for task description
const spanText = document.createElement("span");
spanText.className = "task-text";
spanText.textContent = taskTextValue;

// Create action buttons container
const actionsDiv = document.createElement("div");
actionsDiv.className = "task-actions";

// Create "Complete" button
const completeBtn = document.createElement("button");
completeBtn.className = "btn-complete";
completeBtn.textContent = "Complete";

// Create "Delete" button
const deleteBtn = document.createElement("button");
deleteBtn.className = "btn-delete";
deleteBtn.textContent = "Delete";

// 4. Event Listener: Toggle Completion status
completeBtn.addEventListener("click", () => {
    li.classList.toggle("completed");
    if (li.classList.contains("completed")) {
        completeBtn.textContent = "Completed";
    } else {
        completeBtn.textContent = "Complete";
    }
});

// 5. Event Listener: Delete task item from DOM
deleteBtn.addEventListener("click", () => {
    li.remove();
    updateEmptyState();
});

// Assemble task item DOM structure
actionsDiv.appendChild(completeBtn);
actionsDiv.appendChild(deleteBtn);

li.appendChild(spanText);
li.appendChild(actionsDiv);

// Append to the list and update UI
taskList.appendChild(li);

// Clear input field and refocus
taskInput.value = "";
taskInput.focus();

// Update empty state display
updateEmptyState();

```

```

    }

    // 6. Event Listeners for Adding Tasks
    addTaskBtn.addEventListener("click", addTask);

    taskInput.addEventListener("keydown", (event) => {
        if (event.key === "Enter") {
            event.preventDefault();
            addTask();
        }
    });

    // Initial check on page load
    updateEmptyState();
  });

```

## 7. CODE EXPLANATION

| Code Construct / Element                      | Technical Explanation                                                                               |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>document.createElement()</code>         | Creates a new detached DOM element of the specified HTML tag name.                                  |
| <code>textContent</code>                      | Sets the inner visible text of an element safely without HTML parsing overhead.                     |
| <code>appendChild()</code>                    | Appends a constructed child node as the last child of a parent DOM element.                         |
| <code>addEventListener('click', ...)</code>   | Registers a callback function to execute whenever the element is clicked.                           |
| <code>addEventListener('keydown', ...)</code> | Captures keyboard strokes on the input field to trigger submission upon Enter key.                  |
| <code>classList.toggle('completed')</code>    | Adds the 'completed' class if absent, or removes it if present, altering task styling.              |
| <code>remove()</code>                         | Deletes the target element directly from the DOM tree.                                              |
| <code>style.display</code>                    | Directly alters the element's CSS display property to toggle visibility between 'block' and 'none'. |
| <code>trim()</code>                           | Removes extraneous whitespace from input text to ensure tasks contain meaningful content.           |
| <code>.completed .task-text</code>            | Applies line-through text-decoration and muted text coloring to denote finished tasks.              |

## 8. TEST CASES AND EXECUTION DATA

| Test Case | Input / Action                                   | Expected Result                                          | Actual Result                                    | Status |
|-----------|--------------------------------------------------|----------------------------------------------------------|--------------------------------------------------|--------|
| TC-01     | Load the page in a browser                       | Task list is empty; 'No tasks available.' is shown       | Empty message displayed correctly on load        | Pass   |
| TC-02     | Click Add Task with an empty input               | Browser alert prompt prevents adding empty task          | Alert displayed; no empty item added             | Pass   |
| TC-03     | Type 'Complete Assignment' and click Add Task    | Task appears in the list; empty message hides            | Task added; empty message hidden                 | Pass   |
| TC-04     | Type 'Prepare for Quiz' and press Enter          | Task added on Enter key press identical to button click  | Task added on Enter key press                    | Pass   |
| TC-05     | Add 'Practice JavaScript' task                   | Third task added to list in correct sequential order     | Task appended cleanly to task list               | Pass   |
| TC-06     | Click Complete on a task                         | Task text displays line-through strikethrough styling    | Strikethrough applied to task text               | Pass   |
| TC-07     | Click Complete again on the same task            | Strikethrough is removed (state toggles off)             | Strikethrough removed correctly                  | Pass   |
| TC-08     | Click Delete on a task item                      | Target task item is removed from the DOM list            | Task removed from the DOM cleanly                | Pass   |
| TC-09     | Delete all remaining tasks from the list         | 'No tasks available.' message reappears below empty list | Empty message reappeared after last deletion     | Pass   |
| TC-10     | Hover over Add Task, Complete, or Delete buttons | Button background color and transform animate smoothly   | Smooth hover transitions observed on all buttons | Pass   |

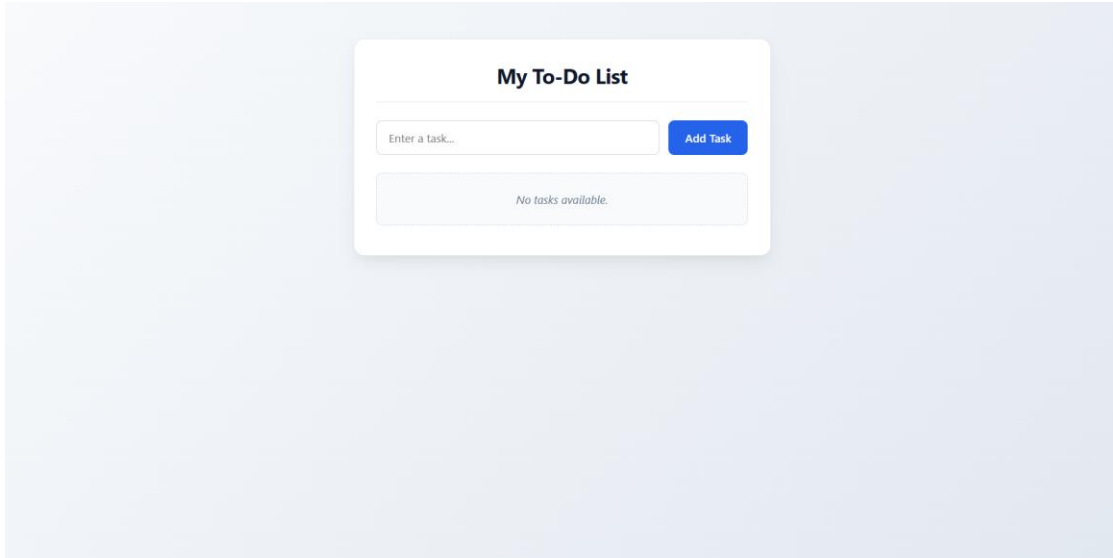
## Test Case Execution Screenshots & Verification Evidence:

### TC-01: Initial Application Load & Empty-State Handling

Action / Input: Load todo.html in browser with an empty task list.

Expected: Heading renders, input field is active, task list has zero items, and 'No tasks available.' placeholder message is displayed.

Observed: Initial load successfully displayed input controls and 'No tasks available.' message. | Status: **PASS**

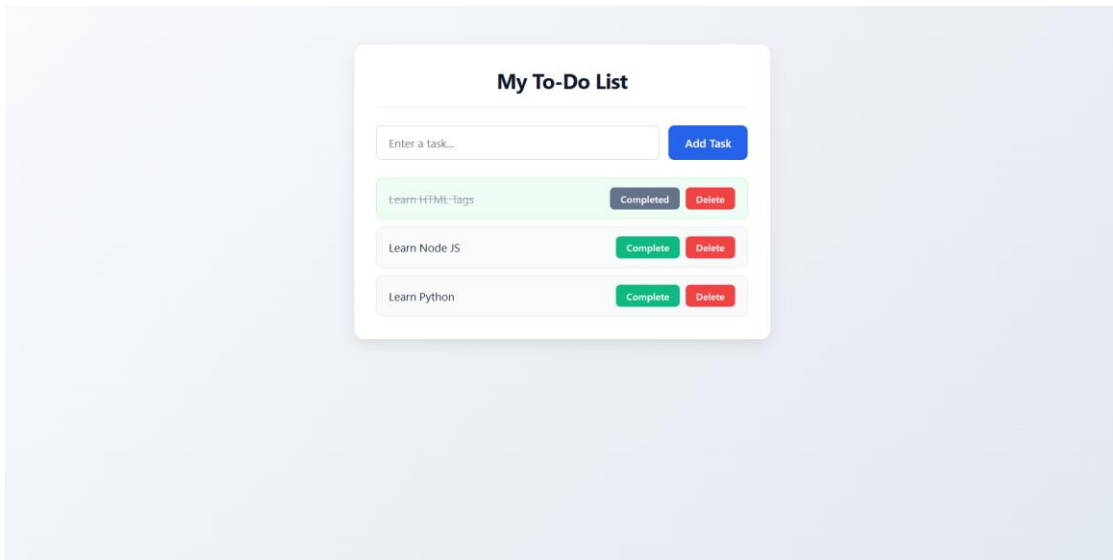


### TC-02: Dynamic Task Addition & Completion State Toggling

Action / Input: Add multiple tasks ('Complete Assignment', 'Prepare for Quiz', 'Practice JavaScript') and click 'Complete' on a task.

Expected: Tasks appended to <ul>, empty message hides, Complete and Delete buttons created dynamically; completed task displays strikethrough styling.

Observed: Tasks listed with action buttons; completed task highlighted with line-through text and toggled button. | Status: **PASS**

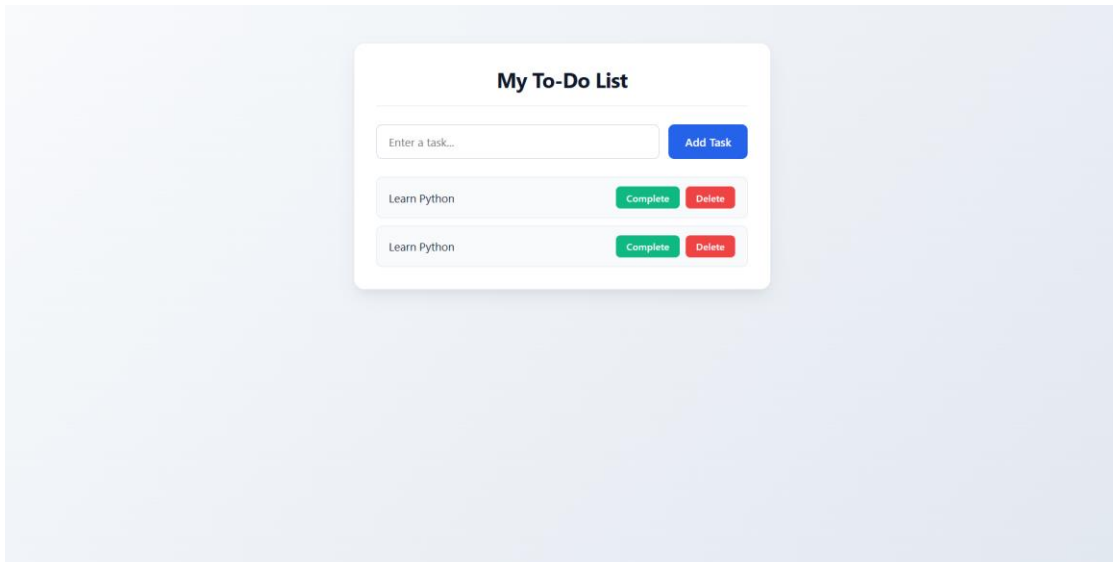


### TC-03: Dynamic Task Deletion & DOM Removal

Action / Input: Click the 'Delete' button on a task item.

Expected: Target `<li>` is removed permanently from DOM; remaining tasks re-index seamlessly in list.

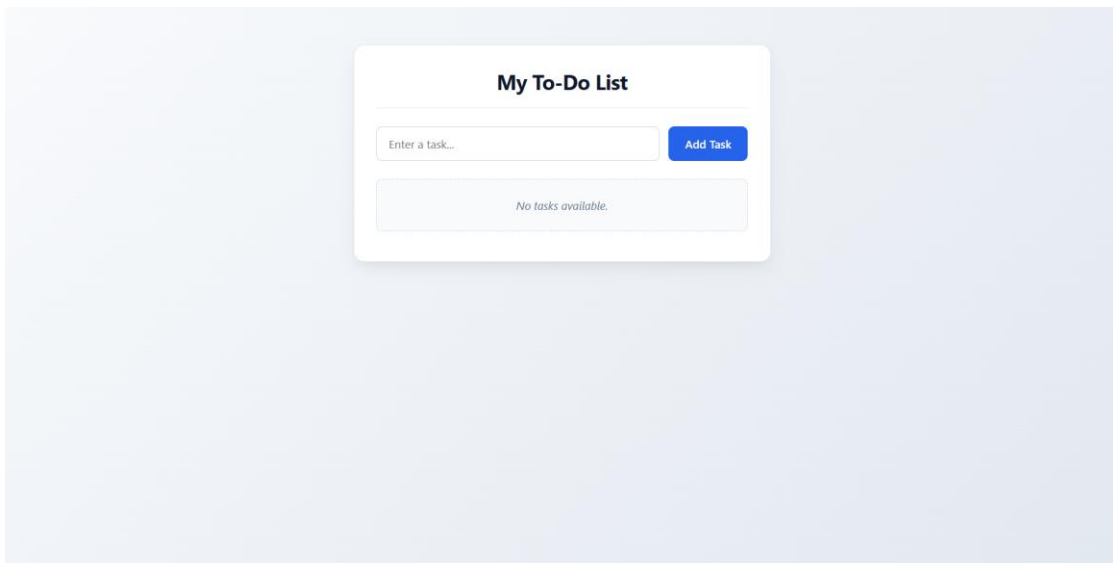
Observed: Task element removed immediately upon clicking Delete; list count updated. | Status: **PASS**



## 9. OUTPUT

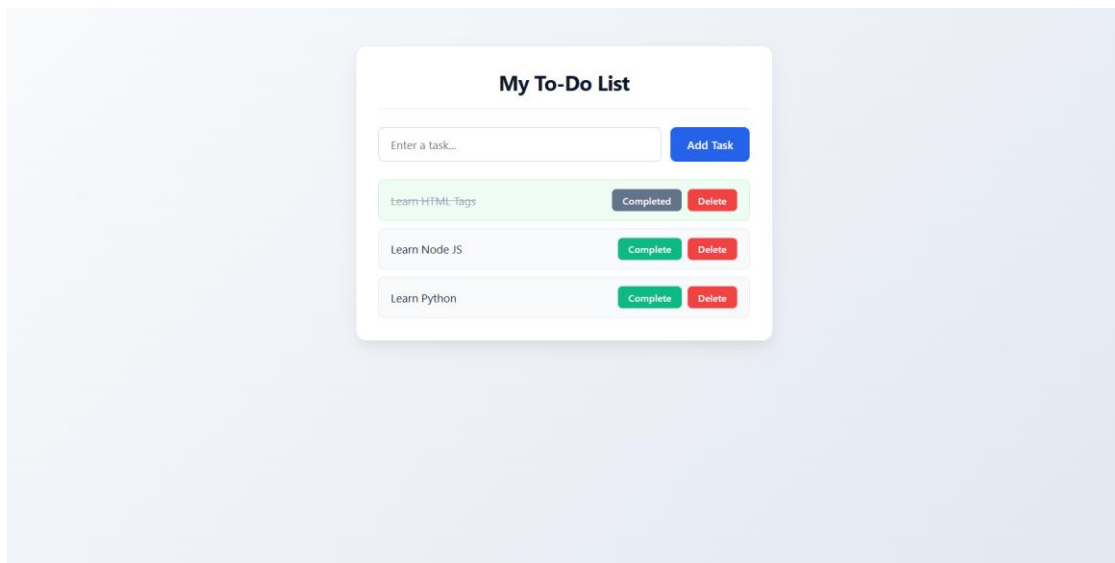
### Output 1: Initial Page Load (Default Empty State)

The initial rendering of the application with an empty task list displaying the 'No tasks available.' placeholder message below the task input area.



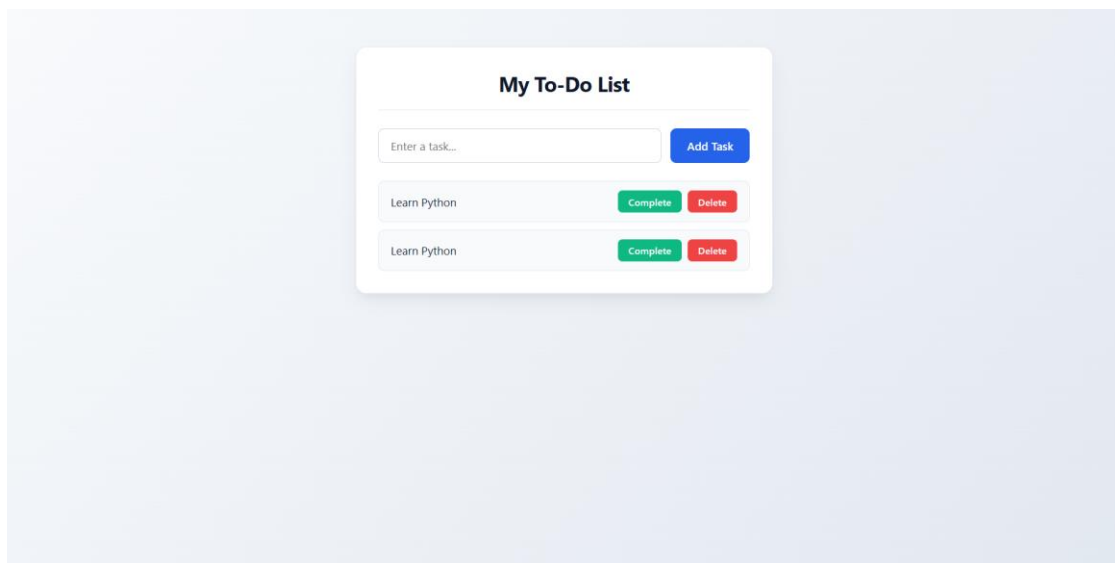
### Output 2: Task List With Multiple Tasks

Demonstration of the active task list after adding tasks ('Complete Assignment', 'Prepare for Quiz', 'Practice JavaScript'), with action buttons and completed strikethrough indication.



### Output 3: Task List After Deleting a Task

Demonstration of DOM removal behavior after clicking the Delete button on a task item, reflecting real-time list updating.



## 10. RESULT

Thus, a dynamic To-Do List web application was successfully designed, developed, and verified using JavaScript DOM manipulation and event listeners. All ten test cases passed successfully, confirming correct task creation, completion toggling, deletion, and empty-state handling with verifiable execution proof.